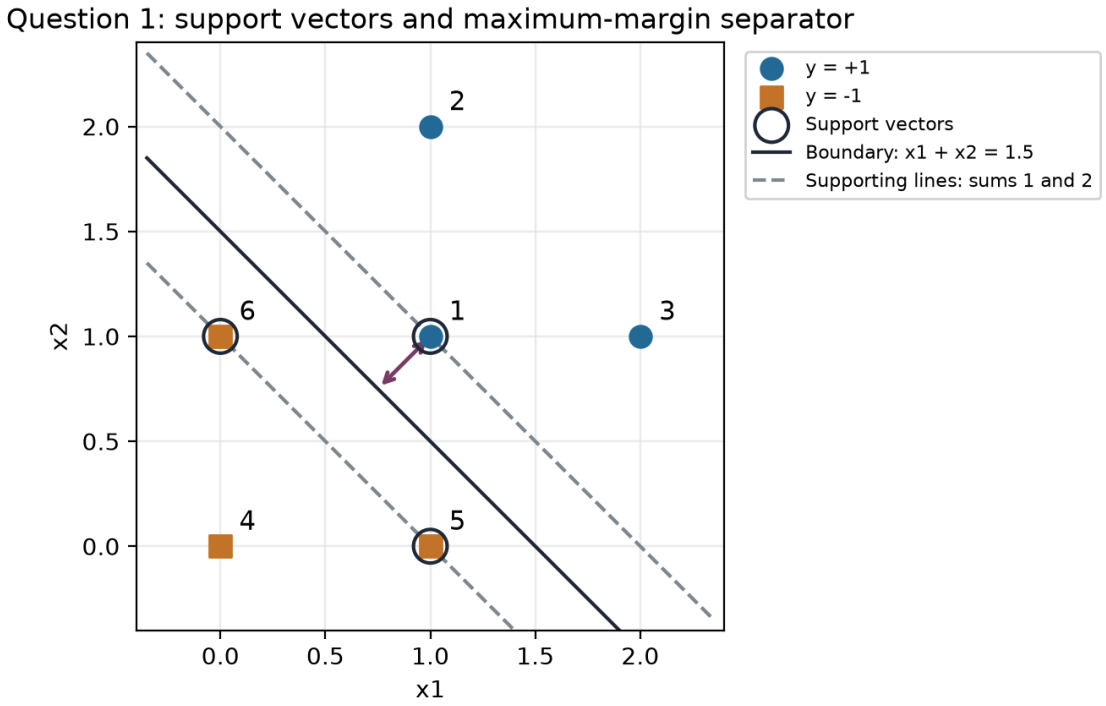
Assignment 3

Question 1(a): plot and support vectors

The positive class has points **1 = (1,1)**, **2 = (1,2)**, and **3 = (2,1)**. The negative class has **4 = (0,0)**, **5 = (1,0)**, and **6 = (0,1)**.



All support vectors are points 1, 5, and 6. By eye, point 1 is the nearest positive point to the negative group. Points 5 and 6 form the exposed negative edge facing it. These points lie on the two parallel supporting lines

$$x_1 + x_2 = 2 \quad \text{and} \quad x_1 + x_2 = 1.$$

The separating line lies halfway between them, at $x_1 + x_2 = 3/2$. The circled points touch the supporting lines; points 2, 3, and 4 lie farther from the separator.

The arrow illustrates the distance from positive support vector 1 to its perpendicular projection $(3/4, 3/4)$ on the decision boundary. The requested one-sided geometric margin is derived on the next page.

Question 1(b): separator, margin, and optimality

Use the canonical hard-margin normalization $y_i(\mathbf{w}^T\mathbf{x}_i + b) \geq 1$. The support vectors satisfy equality. Negative point 5 gives $w_1 + b = -1$, negative point 6 gives $w_2 + b = -1$, and positive point 1 gives $w_1 + w_2 + b = 1$.

The first two equations imply $w_1 = w_2 = a$. Then $a + b = -1$ and $2a + b = 1$, so $a = 2$ and $b = -3$. Therefore,

$$\boxed{\mathbf{w} = (2, 2), \quad b = -3, \quad 2x_1 + 2x_2 - 3 = 0.}$$

The equivalent equation $x_1 + x_2 = 3/2$ describes the same boundary; canonical scaling makes its support-vector functional margins equal to one.

Geometric margin

For any point, its perpendicular distance to the boundary is $|\mathbf{w}^T\mathbf{x} + b|/\|\mathbf{w}\|_2$. Every support vector has numerator one, so

$$\boxed{\gamma = \frac{1}{\|\mathbf{w}\|_2} = \frac{1}{\sqrt{2^2 + 2^2}} = \frac{1}{\sqrt{8}} \approx 0.353553.}$$

This is the **distance from the decision boundary to the nearest training points**. The **full width** between the two supporting lines is separately

$$2\gamma = \frac{1}{\sqrt{2}} \approx 0.707107.$$

Why this margin is maximal

The closest points of the two class convex hulls are positive point $(1, 1)$ and negative-edge midpoint $(1/2, 1/2)$. Their distance is $1/\sqrt{2}$. Any linear separator with one-sided margin γ must place these two hull points at least 2γ apart: signed distance is linear, so a margin constraint holding at all training points also holds throughout each class hull. Thus $\gamma \leq 1/(2\sqrt{2})$. Our boundary attains this bound, proving maximality.

Check against every training point

Point ID	1	2	3	4	5	6
Functional margin	1	3	3	3	1	1

All constraints hold, and equality occurs exactly at the identified support vectors. An independent large-C linear SVM reproduces the derived weights and intercept; it is a check rather than the derivation.

Question 2(a): K-means implementation

For data row $\mathbf{x}_i$ and centroid $\boldsymbol{\mu}_j$, the assignment step is

$$z_i = \arg\min_{0 \le j < k} \|\mathbf{x}_i - \boldsymbol{\mu}_j\|_2^2.$$

The implementation uses NumPy broadcasting and `argmin`. Squaring preserves distance ordering. **Equal-distance ties select the smallest centroid index.**

For each nonempty cluster, update its centroid to the arithmetic mean:

$$\boldsymbol{\mu}_j^{\text{new}} = \frac{1}{|C_j|} \sum_{i \in C_j} \mathbf{x}_i, \qquad C_j = \{i : z_i = j\}.$$

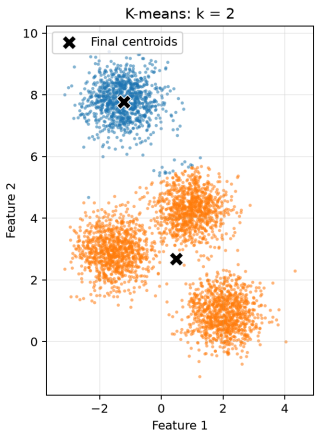
An empty cluster retains its previous centroid. A copy is updated so the input centroid array is unchanged. This policy also handles duplicate initial centroids. No library K-means implementation is used.

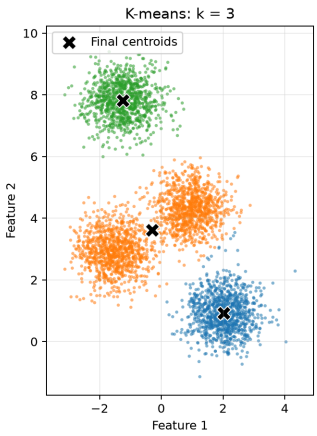
Question 2(b): experiment

The data are `make_blobs(n_samples=4000, centers=4, cluster_std=0.60, random_state=0)`. Initial centroids are sampled **with replacement** using one `default_rng(0)` stream, in the required order $k = 2, 3, 4, 6, 10$. The supplied loop runs 100 assignment/update iterations, followed by a final assignment. True blob labels are discarded before clustering.

Inertia is the sum of squared distances to the assigned centroids. Cluster IDs are arbitrary.

k	Final inertia	Cluster sizes, in index order
2	15222.212	1017, 2983
3	6826.127	1008, 1992, 1000
4	2794.241	1000, 998, 1002, 1000
6	2396.789	1000, 299, 997, 407, 319, 978
10	1914.786	365, 350, 253, 261, 988, 997, 123, 213, 285, 165

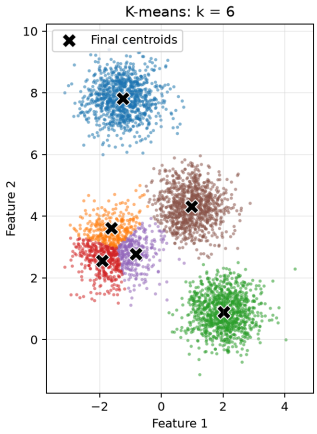


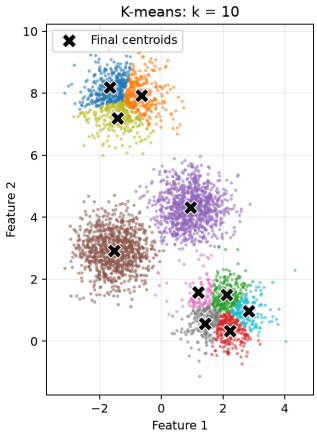


At $k = 2$ or $k = 3$, several generating groups are merged. Axes and limits are identical across all five plots. Black X markers show final centroids.

Question 2(b): increasing the number of clusters







At $k = 4$, the implementation recovers four groups with roughly 1,000 points each, matching the four generating components. Small overlap between nearby Gaussian samples prevents perfectly distinct geometric boundaries.

At $k = 6$ and $k = 10$, additional centroids subdivide some generating groups into smaller regions. A larger k describes finer structure but does not imply that the data were generated by more groups. This run splits the denser nearby groups unevenly, reflecting random initialization and local minima.

The measured inertia decreases from **15,222.212** at $k = 2$ to **1,914.786** at $k = 10$. The globally optimal K-means objective cannot increase when more centroids are allowed, but a single randomly initialized run at each k is not guaranteed to achieve that optimum or a monotone sequence. We report the prescribed runs without a parameter or initialization sweep.

The full-resolution images are also supplied individually under `results/kmeans/`.

Question 3: noisy-label SVM and K-NN repair

Fixed data and paired trials

One NumPy `default_rng(0)` stream generates the training split and then a separate test split. Each has 1,000 positive samples from $N((1,1), I)$ and 1,000 negative samples from $N((-1,-1), 3I)$. The second covariance is $3I$, so each negative feature has standard deviation $\sqrt{3}$, not 3. The two feature sets and the clean test labels remain fixed across all **20** trials.

In each trial, independent uniform draws determine flips from the **original** training labels: probability 0.35 for a positive label and 0.20 for a negative label. One Boolean flip mask is applied once, so no sample can flip twice. Fresh noise is drawn each trial. The baseline SVM uses `kernel="rbf", C=1, gamma=0.01`.

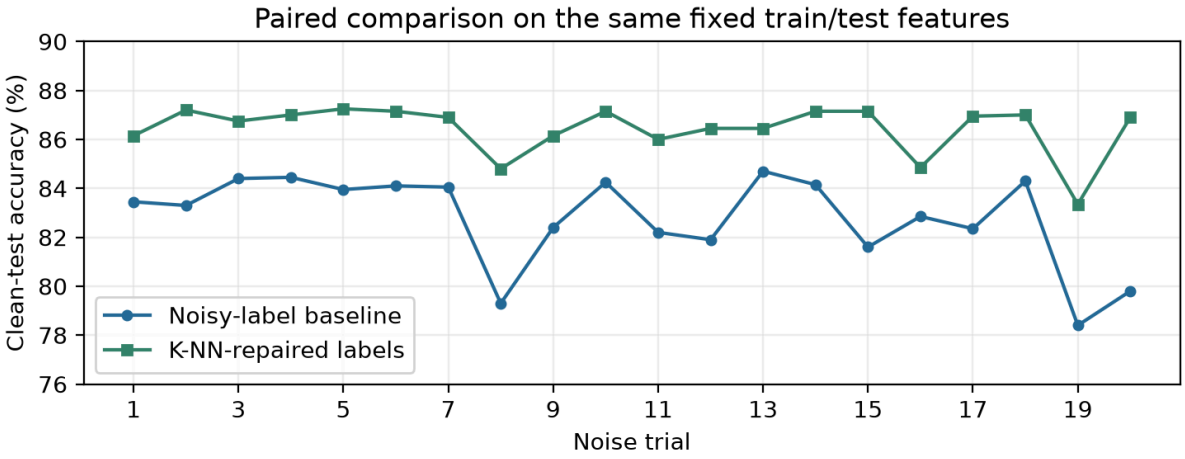
Five-neighbor repair

For each training row, squared Euclidean distances identify its five nearest **other** rows. The diagonal is excluded by row index, even for duplicated coordinates. Stable sorting breaks equal-distance ties by the lower original training-row index. Five binary votes cannot tie. Every vote reads the same unchanged noisy-label array, so repairs are simultaneous. Only training features and noisy labels enter the repair function.

A **fresh SVM with the same parameters** is fitted to the repaired labels. Both methods are evaluated against the same clean test labels, which are used only for evaluation.

Results

Method / comparison	Mean clean-test accuracy
Noisy-label SVM	**82.7950%**
K-NN-repaired-label SVM	**86.4375%**
Repaired minus baseline	**+3.6425 percentage points**



The paired comparison improves in all 20 trials for this seed. These averages describe repeated label corruption on one fixed train/test realization, not accuracy averaged over independent data splits.

Question 3: interpretation and per-trial evidence

K-NN repair helped in this experiment: nearby points often share an underlying class, so majority voting suppresses isolated label flips before SVM fitting. It can also reinforce wrong labels near the overlapping class boundary, erase minority regions, or fail when neighbors are mostly mislabeled.

The asymmetric noise introduces a further bias. If $\eta(\mathbf{x}) = P(Y = +1 \mid \mathbf{x})$, the noisy positive probability is

$$P(\tilde{Y} = +1 \mid \mathbf{x}) = 0.65\eta(\mathbf{x}) + 0.20(1 - \eta(\mathbf{x})) = 0.20 + 0.45\eta(\mathbf{x}).$$

A noisy-label majority therefore favors positive only above $\eta = 2/3$, rather than $1/2$. Local voting can preserve this bias. The observed improvement is empirical, not a guarantee for other seeds or distributions.

All 20 CSV rows

Trial	Baseline accuracy	K-NN accuracy
1	83.45%	86.15%
2	83.30%	87.20%
3	84.40%	86.75%
4	84.45%	87.00%
5	83.95%	87.25%
6	84.10%	87.15%
7	84.05%	86.90%
8	79.30%	84.80%
9	82.40%	86.15%
10	84.25%	87.15%
11	82.20%	86.00%
12	81.90%	86.45%
13	84.70%	86.45%
14	84.15%	87.15%
15	81.60%	87.15%
16	82.85%	84.85%
17	82.35%	86.95%
18	84.30%	87.00%
19	78.40%	83.35%
20	79.80%	86.90%

Reproducibility, checks, and acknowledgment

Run commands and pinned dependencies are in `README.txt` and `requirements.txt`. `verify_solutions.py` checks SVM geometry, K-means updates and edge cases, original-label corruption, and the K-NN repair rules.

Source: the course's Assignment 3 specification and templates. API reference: official scikit-learn SVC documentation. OpenAI Codex generated the solutions, code, report, plots, checks, and packaging. Full acknowledgment and the deidentified visible conversation/tool transcripts are in `ai-disclosure-and-transcripts.zip`. No subagents were used for this assignment.